

CODE READING REPORT

OpenPipe ART 源码架构深度解读与实战教程

面向 Agentic RL / GRPO / 多轮 Rollout / LoRA 训练的工程化阅读指南

Agent Reinforcement Trainer

GRPO

Trajectory-first

vLLM + LoRA

Serverless / Local Backend

源码范式

报告目标：读懂 ART 的整体架构、核心模块职责、源码设计范式，并给出从安装、运行样例到改造成自定义 Agent 训练任务的教程。

资料范围：OpenPipe/ART GitHub 主仓库 README、src/art 核心源码目录、examples、release 页面与公开文档。

抓取时间：2026-08-16；注意 ART 处于快速迭代中，具体 API 以当前安装版本和仓库 main 分支为准。

1. 一句话结论：ART 的核心不是 “Agent 框架”，而是 “让现有 Agent 学会从轨迹中变好”

ART = Agent Reinforcement Trainer。它的定位是把 GRPO / SFT / LoRA / vLLM / 训练后端 / 轨迹采集 / 奖励函数 / 可观测性打包成一个 Python 应用可以接入的训练框架。你的业务代码仍然负责定义任务环境、Agent workflow 和 reward/verifier；ART 负责把这些 rollout 变成可训练的 Trajectory/TrajectoryGroup，再送进本地或 serverless 后端更新模型。

Rollout

Agent 与环境交互

Capture

保存 messages / choices / tools

Reward

写入结果分数

Train

GRPO / SFT 更新 LoRA

Serve

vLLM 加载新权重继续采样

它适合什么问题？

适合	不适合	判断标准
多步工具调用、游戏、搜索、邮箱/浏览器/表格 Agent、数学/代码 verifier 任务、RULER 自动奖励任务	只有一次问答、没有明确评估函数、无法构造环境/奖励、纯聊天风格偏好微调	能否把一次任务执行写成 rollout，并在结束时给出 reward；能否多次并行采样同一类 scenario

2. 项目总览：目录结构与模块边界

examples / notebooks / your_app：定义任务、环境、rollout、reward

↓ 调用

art.Model / art.TrainableModel：模型身份、推理 endpoint、训练入口、日志与成本记录

↓ 产生

Trajectory / TrajectoryGroup：可训练的数据容器，记录消息、choice、reward、metrics、额外历史

↓ 汇总

gather.py / batches.py / preprocessing：并行采样、异常控制、token/policy span、batch 构造

↓ 交给

Backend Protocol：统一本地、Serverless、Megatron、Tinker 等训练后端接口

↓ 执行

local / serverless / vllm_runtime / loss / metrics：训练、推理、checkpoint、可观测性

目录/文件	源码角色	阅读重点
src/art/__init__.py	顶层 API 出口；导出 Backend、TrainableModel、Trajectory、gather、TrainConfig、ServerlessBackend 等	看它导出了什么，就能知道框架主抽象是什么
src/art/model.py	Model 与 TrainableModel；OpenAI 兼容 client proxy；推理元数据、token、cost、policy span 处理	理解“模型作为服务 + 轨迹作为训练数据”的连接点
src/art/trajectories/	Trajectory/TrajectoryGroup、轨迹上下文、序列化、历史、tokenize、tensor 化	ART 的数据模型核心
src/art/gather.py	并发收集 trajectories/groups，记录 reward、completion_tokens、异常率、进度条	理解 rollout 大批量采样如何进入训练循环
src/art/backend.py	Backend Protocol，规定 register、train、_prepare_backend_for_training、exact_adapter_lease 等后端接口	理解本地/云端后端为什么可替换

src/art/local/	本地训练后端、adapter lease、checkpoint、service	本地 GPU 跑 GRPO/SFT 的关键
src/art/serverless/	W&B / serverless 训练后端封装	把训练基础设施外包出去，用户只管环境和奖励
src/art/loss.py	训练损失相关逻辑	GRPO / KL / importance ratio / clipping 等训练指标的根源
src/art/metrics.py + metrics_taxonomy.py	训练、数据、吞吐、队列、成本、vLLM 指标定义	调参和排障时的仪表盘语义
src/art/vllm_runtime.py	vLLM 服务运行配置、路由与推理运行时	模型采样吞吐、LoRA 加载与 inference blocking 相关
src/art/preprocessing/	token metadata、policy span、MoE routing、vLLM tokens	为什么训练需要比普通 OpenAI response 更多元数据
src/art/cli.py	命令行入口，包括 art init、traj-viewer 等能力	快速初始化训练项目与调试轨迹

3. 架构核心：Client-Server + Trajectory-first + Backend Abstraction

3.1 Client-Server 训练环

README 中明确说明 ART 功能分成 client 和 server：client 是 OpenAI-compatible 接口，嵌入你的业务代码；server 在有 GPU 的机器上运行，负责推理和训练循环。推理阶段，用户代码执行 agentic workflow，completion request 路由到 ART server，server 使用 vLLM 运行最新 LoRA；rollout 结束后把 reward 写入 Trajectory。训练阶段，TrajectoryGroup 发给 server，用 GRPO 从最新 checkpoint/LoRA 继续训练，然后保存新 LoRA 并重新载入 vLLM。



3.2 为什么是 Trajectory-first?

传统 SFT 数据是 prompt/answer；Agent RL 数据是多轮交互。ART 把每次完整执行抽象为 Trajectory，而不是单条 completion。这让同一个框架可以支持邮箱搜索、2048、井字棋、MCP、LangGraph、多工具链等任务。

数据层级	含义	典型字段	训练用途
Message / Choice	一次 user/system/assistant 或模型 choice	role、content、tool_calls、usage、token metadata	还原模型在某一步做了什么
Trajectory	一个 scenario 的完整执行轨迹	messages_and_choices、reward、metrics、additional_histories、exchanges	给 GRPO/SFT 提供完整上下文和回报
TrajectoryGroup	同类 scenario 下的一组候选轨迹	list[Trajectory]	GRPO 常需要组内相对 advantage / reward variance
Batch	多个 group 的训练批次	tokenized tensors、mask、policy spans	真正喂给 trainer/loss 的张量

4. 顶层 API 解读：__init__.py 暴露了框架主干

src/art/__init__.py 的第一行 docstring 就把 ART 定义为 “Train LLM agents using RL and SFT”。该文件还做了几个重要工程处理：

- 加载 dotenv，方便使用环境变量。
- 默认压制 LiteLLM serialization warnings，减少长训练日志噪声。
- 清理 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True，因为 torch.cuda.MemPool 的 sleep mode 不支持该配置。
- Unsloth 仅在 IMPORT_UNSLOTH=1 时导入，避免基础 import 过重。
- 尝试给 transformers 打 patch，处理 torchvision 和 preprocess mask 参数问题。
- 集中导出 Backend、Model、TrainableModel、Trajectory、TrajectoryGroup、gather、TrainConfig、TrainSFTConfig、ServerlessBackend、MegatronRuntimeConfig 等。

```
from art import TrainableModel, Trajectory, TrajectoryGroup
from art import gather_trajectories, gather_trajectory_groups
from art import TrainConfig, TrainSFTConfig
from art.serverless import ServerlessBackend
# 这些就是用户脚本最常接触的入口。
```

5. Model / TrainableModel：把“模型身份、推理客户端、训练状态”收在一个对象里

model.py 是 ART 最关键的源码之一。它定义的 Model 是一个 Pydantic BaseModel，字段包括 name、project、entity、run_id、config、trainable，以及 inference_api_key、inference_base_url、inference_model_name 等推理连接信息。源码注释明确说明：Model 可以传给 rollout 函数，也可以用来记录 completions；TrainableModel 是 Model 的子类，可被训练。

Model identity

name / project / entity /
run_id

Inference endpoint

api_key / base_url /
model_name

Training state

trainable / checkpoint /
metrics

5.1 OpenAI Client Proxy 的设计

model.py 里有 `_OpenAIClientProxy`、`_OpenAIChatProxy`、`_OpenAIChatCompletionsProxy`。这不是普通包装，它承担了训练所需的元数据注入：

- 合并 default extra_body，统一给推理请求附加 ART 需要的参数。
- 可选走 binary completions route，用来携带 routed experts 等普通 JSON response 不方便承载的信息。
- 从 vLLM / OpenAI response 中附加 token metadata、completion token metadata、policy token spans。
- 在 policy span mode 开启时禁止 streaming，因为训练需要精确 token span。
- 如果 ART 管理的本地 endpoint 不可达，会抛 `LocalServingUnavailableError`，防止把基础设施错误当作“任务失败数据”。
- 记录推理成本，避免长时间 rollout 对 API/GPU 成本不可见。

设计重点：ART 不是简单地调用 OpenAI API。它要把每次推理 response 增强成“可训练 response”：包含 tokens、policy spans、usage、路由信息、cost 等。这就是为什么 Model 层比一般 Agent SDK 更复杂。

6. Trajectory：Agent RL 的第一性数据结构

ART 的 `src/art/trajectories/` 目录非常大，说明该项目把“轨迹表达、采集、压缩、序列化、tokenize、tensor 化”作为核心工程。该目录包含 `__init__.py`、`_capture`、`_compact.py`、`_compat.py`、`_history.py`、`_protocols.py`、`_scope.py`、`_selection.py`、`_serialization.py`、`_tokenize.py`、`tensors.py` 等。

文件	可推断职责	为什么重要
<code>__init__.py</code>	Trajectory / TrajectoryGroup 的公共定义和上下文管理函数	用户最常直接 import
<code>_capture/</code>	捕获推理调用和消息历史	让业务代码不必手写所有 tracing
<code>_history.py</code>	多轮对话、额外历史、exchange 历史管理	Agent 往往不是单一 chat history
<code>_serialization.py</code>	轨迹序列化/反序列化	保存、回放、可视化、离线训练

_tokenize.py	把多轮消息与 choice 转成可训练 token 序列	训练最复杂的地方：mask、assistant token、tool call token、policy span
tensors.py	tokenized batch 到 tensor 的桥	最终进入 loss/trainer

6.1 Trajectory 的生命周期



7. gather.py：并行 rollout 的“收集器”和“质量阀门”

gather.py 提供 gather_trajectories 与 gather_trajectory_groups。源码逻辑使用 asyncio.gather 并行等待多个 rollout，使用 tqdm 展示进度，通过 GatherContext 统计 reward、completion_tokens、exception 数量，并可用 max_exceptions 控制异常容忍度。

```
groups = await art.gather_trajectory_groups(
    [rollout_group(model, scenario) for scenario in scenarios],
    pbar_desc="collect train rollouts",
    max_exceptions=0.05,
    after_each=optional_filter_or_logger,
)
```

参数	作用	实践建议
pbar_desc	进度条名称	区分 train/val/test rollout
max_exceptions	允许多少任务异常	探索性环境可设 1%-5%；严肃训练建议趋近 0
max_metrics	进度条显示指标上限	指标多时避免 UI 过长
after_each	每个 group 完成后的过滤/日志/再处理	适合做 zero-variance group 丢弃、写 parquet、在线评估

核心设计范式：rollout 是异步、可失败、可并行的；训练数据收集层必须同时处理进度、异常、metrics、token 统计，而不是只返回 list。

8. Backend Protocol：为什么 ART 可以本地训练，也可以 serverless 训练

backend.py 定义的是 Protocol，而不是具体类。它要求后端实现 register、close、_get_step、_prepare_backend_for_training、exact_adapter_lease、train、_train_sft 等接口。这个设计让本地 GPU、W&B serverless、Megatron/Tinker 等后端可以共享用户层 API。

接口	语义	设计原因
----	----	------

register(model)	让 backend 接管模型推理/训练配置	TrainableModel 不需要自己知道本地/云端细节
_model_inference_name	把 ART model 映射到实际 endpoint model name	支持 checkpoint step、LoRA adapter、别名
_prepare_backend_for_training	训练前准备推理/训练服务	vLLM、LoRA、serverless job 都需要准备阶段
exact_adapter_lease	锁定某一步 checkpoint/adapter	避免多 checkpoint/pipeline 训练时采样用错权重
train	后端特定训练入口	不同后端可能有额外参数，不强行统一
_train_sft	SFT 训练流式指标	ART 同时支持 RL 和 SFT warmup

9. types.py：训练配置、结果结构与 Megatron 拓扑

types.py 主要定义轻量类型：OpenAI chat message 类型别名、Tools、TrainConfig、TrainSFTConfig、MegatronTopologyConfig、MegatronRuntimeConfig、TrainResult、LocalTrainResult、ServerlessTrainResult。核心字段如下：

类型	关键字段	解读
TrainConfig	learning_rate=5e-6, kl_penalty_coef=0.0, kl_penalty_source, grad_accumulation_sequences, optimizer_save_interval	RL/GRPO 训练的基础配置；默认学习率较小，偏安全
TrainSFTConfig	learning_rate=5e-5, batch_size='auto', assistant_turns='all'/'last'	SFT warmup 或蒸馏阶段配置
MegatronTopologyConfig	tp, cp, ep, pp, vpp, etp	为更大模型/并行训练准备的拓扑配置
TrainResult	step, metrics	所有 backend 训练结果的父类
LocalTrainResult	checkpoint_path	本地 checkpoint 路径
ServerlessTrainResult	artifact_name	云端/W&B artifact 名称

10. Local Backend：本地训练时实际承担“GPU 服务编排”

src/art/local/ 包含 backend.py、adapter_leases.py、checkpoints.py、service.py。其中 local/backend.py 体量很大，是本地后端主实现。根据目录与 Backend Protocol 的关系，它主要负责：注册模型、启动/管理本地推理服务、准备训练、保存/清理 checkpoint、管理 adapter lease、执行 RL/SFT 训练并返回 LocalTrainResult。

checkpoints.py

保存/发现/清理 checkpoint，决定 step 与路径语义

adapter_leases.py

训练与推理并发时保证 adapter 使用一致性

service.py

本地服务生命周期/辅助服务入口

backend.py

组合训练、推理、日志、权重更新与资源管理

11. Serverless Backend：把训练基础设施变成“服务”

README 的 Serverless RL 示例使用 `from art.serverless.backend import ServerlessBackend`，然后创建 `TrainableModel` 并 `model.register(backend)`。它的目标是让用户不再自己管理 GPU、vLLM、checkpoint 和基础设施健康，只定义 data、environment 和 reward function。

```
import art
from art.serverless.backend import ServerlessBackend

model = art.TrainableModel(
    project="voice-agent",
    name="agent-001",
    run_name="agent-001",
    base_model="Qwen/Qwen3.6-27B",
)
backend = ServerlessBackend(api_key="your_wandb_api_key")
model.register(backend)
```

实践判断：如果你在公司里只是验证 reward 和 task 是否有效，serverless 更省工程时间；如果你要研究训练系统、改 loss、改 tokenizer/tensor pipeline、接自研 vLLM/SGLang，则本地 backend 更适合。

12. loss / preprocessing / metrics：真正决定训练是否稳定的三件事

模块	关键问题	源码设计意图
loss.py	reward advantage、KL、importance ratio、clipping、entropy、grad norm	把 trajectory tensor 变成稳定的 GRPO/RL 更新
preprocessing/ vllm_tokens.py	response 是否有准确 token 数和 token id	训练需要知道哪些 token 是模型生成的

preprocessing/ policy_spans.py	哪些 token 属于哪个 policy/checkpoint	pipeline/multi-checkpoint/adapters 场景防止 off-policy 混乱
preprocessing/ moe_routing.py	MoE 模型 routed experts 元数据	为 MoE 训练与诊断保留专家路由信息
metrics.py	训练、吞吐、队列、成本、vLLM、数据效率指标	让训练问题能被定位，而不是只看 reward

12.1 你应该重点盯的指标

指标族	说明	坏信号
objective/reward	训练目标是否提升	长期不动或剧烈震荡
loss/kl_div 或 kl_policy_ref	模型偏离参考策略程度	过快上升代表可能跑飞
loss/importance_ratio_p95/p99	新旧策略分布差异	尾部过大代表 off-policy 风险
discarded/stale_groups	过期 group 被丢弃	rollout 太慢或 pipeline 太深
throughput/*tok_per_s	训练/推理吞吐	GPU 在等环境或环境在等 GPU
vllm/prefix_cache_hit_rate	前缀缓存命中	低命中代表 prompt 结构可能无法复用

13. 如何使用：从安装到跑通第一个样例

13.1 安装

```
# 方式一：直接安装
pip install openpipe-art

# 方式二：源码开发
# git clone https://github.com/OpenPipe/ART.git
# cd ART
# uv sync

# ART 还提供 AI coding assistant 初始化技能
uv run art init
```

硬件上，如果使用 local backend，需要 CUDA GPU；如果使用 ServerlessBackend，则本地只需能运行 Python 客户端和你的环境逻辑。

13.2 最小训练脚本骨架

```
import asyncio
import art
from art.serverless.backend import ServerlessBackend

async def rollout(model: art.Model, task) -> art.Trajectory:
    client = model.openai_client()
    traj = art.Trajectory(messages_and_choices=[], reward=0.0)
    # 1. 构造 messages
    # 2. 调用 client.chat.completions.create(...)
    # 3. 执行环境动作 / 工具调用
    # 4. 根据 verifier 写入 traj.reward 和 traj.metrics
    return traj

async def main():
    model = art.TrainableModel(
        project="my-agent",
        name="agent-001",
        base_model="Qwen/Qwen3.6-27B",
    )
    backend = ServerlessBackend(api_key="YOUR_WANDB_KEY")
    await backend.register(model) # 或 model.register(backend), 视版本 API 而定

    for step in range(20):
        groups = await art.gather_trajectory_groups([
            make_group(model, scenario) for scenario in sample_scenarios()
        ])
        await backend.train(model, groups, config=art.TrainConfig())

asyncio.run(main())
```

注意：上面是模式化骨架。ART 版本迭代很快，具体 register/train 调用形式要以你安装版本的 examples 和 docs 为准。源码阅读的重点是理解对象分工：model 负责推理身份，rollout 负责产生 trajectory，backend 负责训练。

14. 典型用例教程：如何把自己的 Agent 接进 ART

14.1 游戏类：2048 / Tic Tac Toe / Codenames

步骤	做法	例子
环境	写一个可重置、可执行动作、可判断终局的 Python 环境	棋盘、2048 grid、合法动作集合
Prompt	把当前状态转成模型能理解的文本	“当前棋盘如下，请输出 move=left/right/up/down”
动作解析	解析模型输出并执行	非法动作记 penalty 或重试
Reward	终局分数 + 中间奖励	胜负、分数增长、步数惩罚
TrajectoryGroup	同一个 initial state 采样多个候选	GRPO 用组内表现差异学习

14.2 检索/邮箱 Agent：ART-E 范式

ART-E 的关键不是“会调用搜索工具”，而是构造一个评估闭环：给定邮件库和问题，Agent 多轮搜索、打开邮件、提取证据，最终回答；verifier/RULER 判断答案是否正确。这个范式适合公司内部知识库、客服工单、CRM、科研论文库。



14.3 MCP / LangGraph Agent

README 显示 ART 近期示例覆盖 MCP-RL 和 LangGraph 集成。这类任务的做法是：保留原 LangGraph/MCP workflow，替换/包裹模型调用，让 ART 捕获每次模型决策和工具调用结果；reward 可以来自任务成功率、工具调用正确率、终态校验或 RULER。

14.4 数学/代码类任务

数学和代码任务的优势是 verifier 明确。数学可用 exact match、sympy、单元测试；代码可用 pytest、hidden tests、执行沙箱。建议先用小模型 + 小数据验证 reward 是否有效，再扩大 batch 和模型尺寸。

15. 从 0 到 1 设计自定义 ART 项目的模板

阶段	要产出的东西	检查标准
任务定义	scenario schema	每个 scenario 可独立重放
环境封装	reset/step/render/is_done	非法动作有明确定义
模型接口	Model/TrainableModel + openai_client	所有模型调用被 ART 捕获
rollout	async rollout(model, scenario) -> Trajectory	异常可控，reward 必填
分组	同一 scenario 多次采样形成 group	组内 reward 有方差
训练	backend.train + TrainConfig	reward 不跑飞，KL/ratio 可控
评估	固定 test scenarios	训练集提升不等于泛化提升
可观测	metrics / trajectory viewer / W&B	能定位失败轨迹

16. 设计范式总结：ART 源码背后的 8 个工程原则

1. Trajectory-first
把 Agent 训练数据建模为完整执行历史，而不是单轮问答。

2. OpenAI-compatible boundary
业务代码仍像调用 OpenAI 一样调用模型，底层可替换为 ART 管理的推理服务。

3. Backend abstraction

训练基础设施隐藏在 Backend Protocol 后，本地和云端共享上层范式。

4. Metadata-rich response

普通 response 不够训练，必须附加 token、policy span、usage、routing 等。

5. Async rollout

Agent RL 的瓶颈常在环境/工具调用，因此并行收集和异常控制是一等公民。

6. Group-based learning

GRPO 强调同一 scenario 下多个样本的相对表现。

7. Observability built-in

不只看 reward，还看 KL、ratio、stale group、吞吐、成本和 vLLM 指标。

8. Small API, heavy internals

用户层 API 简单，源码内部承担复杂训练系统工程。

17. 与 VeRL / TRL / 普通 Agent 框架的区别

项目类型	优势	不足	ART 的差异点
普通 Agent 框架	工具编排方便	通常不训练权重	ART 关注从 rollout 中更新模型
TRL	训练算法成熟	Agent 环境接入成本较高	ART 封装 Agent rollout 到 GRPO 的桥
VeRL	分布式 RL infra 强	工程门槛更高	ART 更偏应用开发者友好
自研训练脚本	可控性强	成本、checkpoint、服务、metadata 都要自己写	ART 抽象了训练服务器和推理服务

18. 常见坑与排障

问题	症状	处理建议
reward 设计太稀疏	大部分轨迹都是 0 分，训练不动	加中间奖励或 RULER 分级评分
TrajectoryGroup 零方差	组内 reward 全一样	提高采样温度、扩大 group size、改 scenario 难度
环境太慢	GPU 等 rollout	异步工具调用、缓存环境、并行 scenario
off-policy 太严重	importance ratio p95/p99 飙升	减少 rollout 队列滞后，缩短采样到训练延迟

模型输出不可解析	非法动作多	强格式 prompt、JSON schema、guided completion、非法动作 penalty
训练跑飞	reward 短升后崩、KL 大	降低学习率、增加 KL penalty、缩小每步训练量
本地服务不可达	LocalServingUnavailableError	检查 vLLM/端口/GPU 内存，而不是把 rollout 当作失败样本

19. 推荐阅读源码顺序

1. **README.md**: 先理解 Client-Server 训练循环和官方示例。
2. **examples/2048**、**tic-tac-toe**、**ART-E**、**MCP-RL**: 看一个完整任务如何定义 rollout/reward。
3. **src/art/__init__.py**: 确认顶层 API。
4. **src/art/model.py**: 理解 Model、TrainableModel、OpenAI proxy。
5. **src/art/trajectories/__init__.py**: 理解 Trajectory/TrajectoryGroup。
6. **src/art/gather.py**: 理解并行采样和异常指标。
7. **src/art/backend.py** + **src/art/local/backend.py**: 理解训练后端协议与本地实现。
8. **src/art/loss.py**、**preprocessing**、**metrics**: 深入训练稳定性与数据转换。

20. 一周复现路线

天数	目标	产出
Day 1	读 README + 跑官方 notebook 或 2048	能解释 rollout->reward->train
Day 2	读 Model/Trajectory/Gather	画出自己的数据流图
Day 3	改一个游戏环境 reward	观察 reward 和 KL 变化
Day 4	写一个小型数学 verifier 任务	完成自定义 scenario/rollout
Day 5	加入 W&B/trajectory viewer	能回放失败轨迹
Day 6	对比 SFT warmup + RL vs 纯 RL	形成训练曲线
Day 7	迁移到你的真实 Agent 任务	确定环境、奖励、预算、评估集

21. 资料来源与核验说明

主要来源: OpenPipe/ART GitHub README; OpenPipe/ART GitHub Releases; src/art 目录源码, 包括 __init__.py、model.py、backend.py、gather.py、types.py、src/art/trajectories、src/art/local; 公开文档 art.openpipe.ai; OpenPipe 官方博客。GitHub 页面显示该项目定位为 “Agent Reinforcement Trainer: train multi-step agents for real-world tasks using GRPO”, README 中说明其训练循环由 inference 与 training 两阶段组成, 包含 vLLM 最新 LoRA、Trajectory reward、GRPO 训练、checkpoint 保存与重新加载。

注意: ART 更新频率较高。本报告注重架构和源码范式, 具体函数签名、参数名、CLI 子命令, 应以你安装的 openpipe-art 版本、仓库 main 分支、examples 与 docs 为准。